

Course Introduction and the AI Engineering Mindset

Turning formal algorithms into correct, testable, production-quality systems

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Distinguish an AI-theory claim from an AI-engineering deliverable.
- 02 — Translate formal pseudocode into data structures, invariants, and tests.
- 03 — Explain how hidden tests and performance limits change implementation choices.
- 04 — Select fail-fast or fail-silent behavior deliberately for a stated contract.
- 05 — Apply the course generative-AI policy to realistic development scenarios.
- 06 — Plan effort across the course's modules, projects, and examinations.

The unit of success changes in an engineering course

A correct idea becomes an engineering result only when its behavior is specified, implemented, and independently verified.

Evidence

Correctness must be observable beyond the author's favorite example.

- A proof addresses a model; software must also honor an executable contract.
- Representations and interfaces determine which theoretical assumptions remain true.
- Tests, complexity measurements, and failure behavior are part of the result.

Theory, implementation, and verification form one pipeline

Theory

What follows from the formal assumptions?

- Define the model and prove properties.
- Derive an update or decision rule.
- Analyze asymptotic behavior.
- State conditions for guarantees.

Implementation

How will the model exist in software?

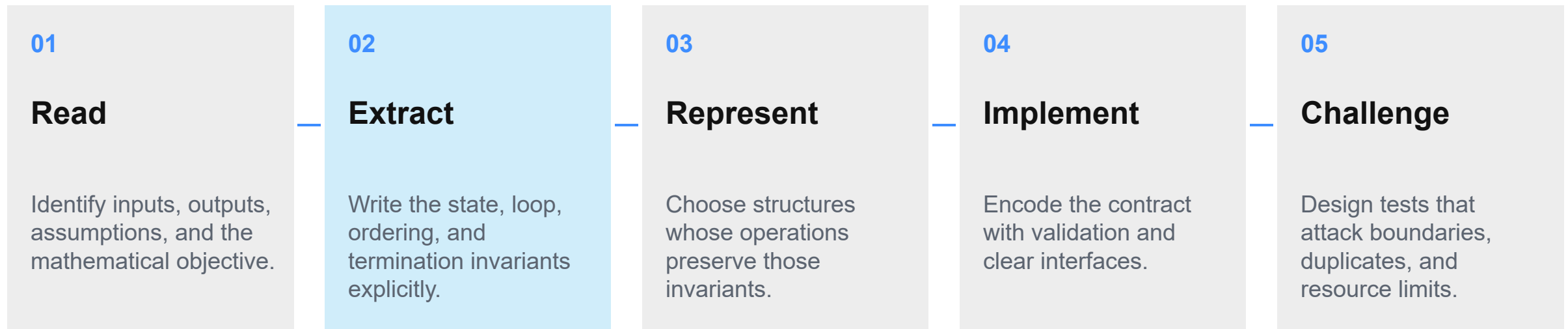
- Choose representations and interfaces.
- Preserve invariants under mutation.
- Handle invalid and degenerate inputs.
- Control time, space, and numerical cost.

Verification

What evidence would reveal a defect?

- Design unit and property tests.
- Exercise boundaries and adversarial cases.
- Measure complexity-sensitive behavior.
- Reproduce failures deterministically.

Translate pseudocode through invariants, not line by line



The invariant is the bridge between a formal algorithm and trustworthy code.

Pseudocode leaves engineering decisions unresolved

```
function GRAPH_SEARCH(problem):  
    frontier <- { NODE(problem.initial) }  
    explored <- empty set  
  
    while frontier is not empty:  
        node <- REMOVE(frontier)  
        if problem.goal_test(node.state):  
            return SOLUTION(node)  
        add node.state to explored  
        for child in EXPAND(problem, node):  
            if child.state not in explored or frontier:  
                add child to frontier  
  
    return failure
```

Questions the code must answer

- What ordering does REMOVE enforce?
- Are state keys immutable and hashable?
- When is a cheaper duplicate updated?
- What precise value represents failure?

Correctness is a conjunction, not a screenshot

correct result = contract satisfied AND invariants preserved AND termination justified

- Contract: the observable relation between valid inputs, outputs, and errors.
- Invariant: a property that remains true at every relevant program point.
- Termination: either a valid result or a specified failure occurs in finite work.

ENGINEERING IMPLICATION

A passing example establishes none of these conditions by itself.

An autograder is an independent executable specification

Hidden tests ask whether your implementation obeys the contract when the author no longer controls the example.

Independence

The evaluator chooses inputs, scale, and failure conditions.

- Public tests teach the interface; hidden tests probe generality.
- Timeouts make complexity and data-structure choices observable.
- Deterministic outputs make failures reproducible and gradable.

Public examples start the test design; they do not finish it

Visible evidence

- Nominal example from the assignment.
- Expected function signature and return type.
- One representative path or numerical result.
- A local smoke test for the submission workflow.

Independent probes

- Empty, singleton, and malformed inputs.
- Cycles, duplicates, ties, and unreachable goals.
- Large instances that expose complexity.
- Repeated runs that expose nondeterminism.

A zero-length solution exposes a search-agent defect

SCENARIO

A start state already satisfies the goal test, but the implementation expands successors before checking the goal.

- The returned path may contain an unnecessary move.
- A state with no legal actions may incorrectly report failure.
- The algorithm violates the shortest-path contract for zero-length solutions.
- A direct unit test should assert an empty action sequence and zero cost.

Engineering fix: check the initial node against the goal contract before expansion, then lock the behavior with a regression test.

Defensive programming makes assumptions executable

Validate the boundary once, fail with useful information, and keep the trusted core simple.

Contract

Invalid data should not drift into the algorithm as corrupted state.

- Check shapes, ranges, types, and domain constraints at public interfaces.
- Use precise exceptions or documented sentinel values, never accidental behavior.
- Avoid redundant checks inside a core whose preconditions are already established.

Choose failure semantics from the system contract

Fail fast

- Reject invalid developer input immediately.
- Raise a precise exception near the boundary.
- Protect invariants from corrupted internal state.
- Best when silent recovery would hide a defect.

Fail safely

- Return a documented degraded result.
- Preserve service continuity when policy allows.
- Emit monitoring evidence for later diagnosis.
- Best when a safe fallback is part of the contract.

The AI-use policy protects the learning objective

Permitted

Support understanding without replacing the core work.

- Explain an error message.
- Clarify Python or NumPy syntax.
- Discuss a concept abstractly.
- Suggest test categories after your design.

Prohibited

Outsource the algorithm students must learn to construct.

- Generate the assignment's core algorithm.
- Produce a submission-ready implementation.
- Rewrite hidden-test failures into a solution.
- Present generated reasoning as personal work.

Your responsibility

Be able to explain, defend, and reproduce every submitted decision.

- Follow the assignment-specific policy.
- Disclose assistance when required.
- Verify every factual and technical claim.
- Ask before using a borderline workflow.

Policy checkpoint

CHECKPOINT

A student writes BFS, then asks an AI system to explain why a cyclic graph causes a timeout and to list possible test cases. Is that use permitted?

01

Identify the assignment's core learning objective.

02

Separate diagnosis from algorithm generation.

03

State what the student must verify independently.

Three modules expand one engineering discipline

01

Agents

Model goals, state, actions, search, and adversarial decisions.

02

Uncertainty

Reason with MDPs, reinforcement learning, and Bayesian belief.

03

Learning

Fit, evaluate, regularize, and audit predictive models.

04

Integration

Select methods by assumptions, evidence, and deployment risk.

The algorithms change; contracts, invariants, tests, and responsible evidence remain.

Assessment rewards both repeated practice and integration

Weekly work - 15%

Code labs and quizzes create short feedback cycles.

- Practice before integration.
- Treat tests as design feedback.
- Repair misconceptions early.

Module projects - 30%

Three projects combine each module's full algorithmic arc.

- Integrate multiple components.
- Defend design decisions.
- Demonstrate robust behavior.

Examinations - 50%

The source lists a 30% midterm category and a 20% final.

- Trace and derive without a codebase.
- Explain guarantees and failure modes.
- Transfer ideas to unfamiliar cases.

Passive familiarity will not survive an unfamiliar test

Prepare by constructing, tracing, breaking, and explaining systems - not by rereading until the notation looks familiar.

- Trace algorithms on small examples before coding.
- Write tests from the contract before seeing a failure.
- Explain guarantees and counterexamples without notes.

Practice

Retrieval and implementation expose gaps that recognition conceals.

Key takeaways

- 01 — **AI engineering connects formal guarantees to executable contracts and independent evidence.**
- 02 — Invariants guide representation, implementation, and test design.
- 03 — Autograders expose generality, complexity, determinism, and edge-case behavior.
- 04 — Responsible tool use preserves the reasoning that an assignment is designed to assess.

Next: Rational agents, PEAS descriptions, and formal search problems

Check your understanding

Q1

Why can a theoretically correct algorithm still be an incorrect engineering submission?

Q2

Name one invariant and one edge-case test for a graph-search implementation.

Q3

How does a timeout communicate a requirement beyond functional correctness?

Q4

When is a safe fallback preferable to an exception, and what observability is required?

Q5

Classify one permitted and one prohibited use of generative AI in a search lab.